Computer Organisation and Architecture
Assignment 2

FSM-Based Direct-Mapped Cache Controller Simulation Report

A cycle-accurate simulation of a direct-mapped write-back cache controller implemented as a Finite State Machine in Python

Language:	Python 3
Policy:	Write-Back + Allocate-on-Write
Cache:	4-line Direct-Mapped, 4 words/block
Address:	8-bit (Tag[4] · Index[2] · Offset[2])
Memory Latency:	3 cycles
Submission Date:	16 April 2026

Submitted by:

ID: 230041227

ID: 230041137

ID: 230041233

GitHub Repository: https://github.com/Abid3301/Simple-FSM

Contents

1 System Architecture Overview **2**

2 Address Format & Cache Organisation **2**

2.1 Address Decoding Examples 3

2.2 Cache Line Structure 3

3 Finite State Machine Design **3**

3.1 FSM State Diagram 4

3.2 State Descriptions 4

3.2.1 STATE: IDLE 4

3.2.2 STATE: COMPARE_TAG 4

3.2.3 STATE: ALLOCATE 5

3.2.4 STATE: WRITE_BACK 5

4 Signal Interface **5**

5 Core FSM Tick Logic **5**

5.1 Core FSM Tick Logic 5

6 Simulation Scenarios **6**

6.1 Scenario 1: Read Miss – Cold Start 6

6.2 Scenario 2: Spatial Locality — Read Miss then Hits 7

6.3 Scenario 3: Write Hit — Dirty Bit Confirmed by Read-Back 8

6.4 Scenario 4: Write Miss — Allocate-on-Write 9

6.5 Scenario 5: Dirty Eviction — WRITE_BACK Path 9

6.6 Scenario 6: Mixed Read/Write Stress Test 10

7 Conclusion **12**

1. System Architecture Overview

The simulation models three interacting components that together recreate the memory hierarchy found in a typical single-core processor: a **CPU** that issues read/write requests, a **cache controller FSM** that manages a small direct-mapped cache and mediates between CPU and memory, and a **main memory** that is always available but introduces a multi-cycle latency for block transfers.

Table 1: System Components at a Glance

Component	Description
CPU (Request Queue)	Issues `READ` / `WRITE` requests to the cache controller; stalls until acknowledged by the controller.
Cache Controller FSM (4 States)	**`IDLE`** · **`COMPARE_TAG`** · **`ALLOCATE`** · **`WRITE_BACK`**. Arbitrates between the CPU and main memory.
Main Memory (256 Words)	Never misses; a block read or write costs exactly 3 cycles (`MEM_LATENCY = 3`).
Cache (4 × 4)	4 lines, 4 words per block; each line carries a *valid* bit, a *dirty* bit, a *tag*, and 4 data words.

Each component is clocked synchronously with every call to `CacheController.tick()` advances simulation time by exactly one cycle. The CPU drives the FSM by depositing a request object at the start of each cycle; the FSM replies through output signals (`cpu_stall`, `cpu_dout`, `hit`, `mem_read`, `mem_write`) that are sampled by the CPU at the *end* of the same cycle. This models the classic synchronous handshake described in Patterson & Hennessy.

Design Choice — Write-Back + Allocate-on-Write

On a write miss the controller *first* fetches the target block from memory (**`ALLOCATE`**), *then* modifies the word in cache and sets the dirty bit. The modified block is only written back to memory when it is evicted by a conflicting miss. This maximises bandwidth efficiency and matches the behaviour described in the textbook's "simple cache" example.

2. Address Format & Cache Organisation

All addresses are 8-bit. Three fields are decoded by `decode(addr)` at the moment a new request arrives in **`IDLE`**:

```
  7   6   5   4   3   2   1   0
+---------------+-------+-------+
|   TAG [7:4]   | IDX   |  OFF  |
```

```
|    4 bits     | [3:2] | [1:0] |
|               | 2 bit | 2 bit |
+---------------+-------+-------+

Tag    = addr >> 4              (selects which block is cached)
Index  = (addr >> 2) & 0x3     (selects which cache line to look in)
Offset = addr & 0x3            (selects word within block)
Block base = addr & ~0x3       (aligned address sent to memory)
```

Listing 1: 8-bit Address Field Decomposition

2.1. Address Decoding Examples

Table 2: Sample Address Decoding

Address	Binary	Tag	Index	Offset	Block Base
0x00	0000 0000	0x0	0	0	0x00
0x05	0000 0101	0x0	1	1	0x04
0x09	0000 1001	0x0	2	1	0x08
0x40	0100 0000	0x4	0	0	0x40
0x44	0100 0100	0x4	1	0	0x44

The direct-mapped organisation means that addresses `0x00`, `0x10`, `0x20`, ..., `0x40` all map to **index 0** with different tags, accesses to any two of these within the same short window will cause *conflict misses*. This is deliberately exercised in Scenarios 5 and 6.

2.2. Cache Line Structure

```python
@dataclass
class CacheLine:
    valid : bool      = False    # line contains meaningful data
    dirty : bool      = False    # data differs from memory copy
    tag   : int       = 0        # upper address bits
    data  : List[int] = [0,0,0,0]  # 4 words (bytes in our model)
```

Listing 2: `CacheLine` Dataclass

3. Finite State Machine Design

The controller is a four-state Moore/Mealy hybrid FSM. Outputs are partly state-dependent (Moore) and partly transition-dependent (Mealy). The four states model the three phases of

cache management: idle waiting, tag comparison, memory-block fetch, and dirty-block eviction.

3.1. FSM State Diagram

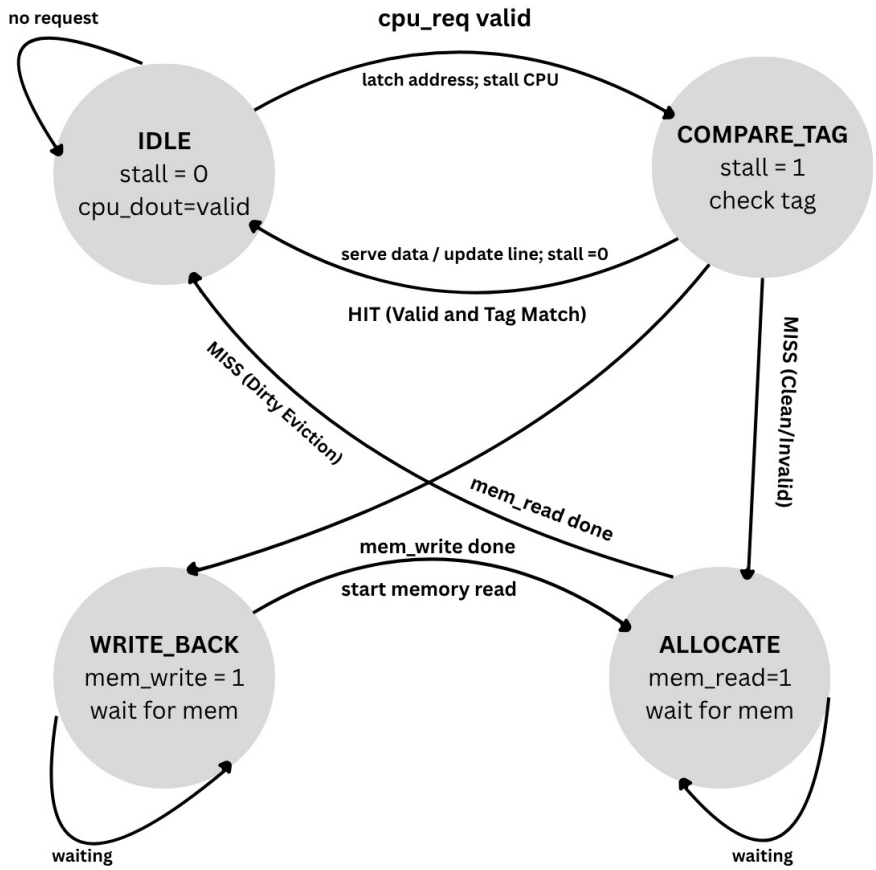

Figure 1: Four-State FSM of the Direct-Mapped Write-Back Cache Controller

3.2. State Descriptions

3.2.1. STATE: IDLE

The controller is quiescent. `cpu_stall` is deasserted. On the cycle a new CPU request arrives the address is decoded, the fields are latched into `cur_tag` / `cur_idx` / `cur_off`, `cpu_stall` is asserted, and the FSM transitions to **`COMPARE_TAG`**. Without a request the machine loops back to **`IDLE`** with all signals low.

3.2.2. STATE: COMPARE_TAG

The tag and valid bit of `cache[cur_idx]` are checked. A **HIT** occurs when `valid == True` and `cache_line.tag == cur_tag`. On a read hit the requested word is placed on `cpu_dout` and `stall` is cleared. On a write hit the word is updated in-place and the **dirty** bit is set. Both hit

paths return immediately to **`IDLE`** in the *same* cycle. A **MISS** on a clean/invalid line transitions to **`ALLOCATE`**; a miss on a dirty line transitions to **`WRITE_BACK`**.

3.2.3. STATE: ALLOCATE

A block read has been issued to memory. The FSM spins in this state, asserting `mem_read`, until `Memory.tick()` returns `done=True` after `MEM_LATENCY` cycles. When done: the cache line is filled (`valid=1, dirty=0, tag=cur_tag, data=block`). If the original request was a write, the target word is immediately updated and `dirty` is set. The CPU stall is then cleared and the FSM returns to **`IDLE`**.

3.2.4. STATE: WRITE_BACK

The evicted dirty block must be flushed to memory before the new block can be fetched. The old block's memory address is reconstructed from `line.tag` and `cur_idx`:

$$\text{old_base} = (\text{line.tag} \ll 4) \mid (\text{cur_idx} \ll 2)$$

The FSM spins here asserting `mem_write` until memory acknowledges completion, then issues a read for the new block and moves to **`ALLOCATE`**.

4. Signal Interface

The following table summarises every signal in the simulation and its active state for each FSM state.

Table 3: Signal Truth Table by FSM State

Signal	Direction	IDLE	CMP_TAG	ALLOCATE	WRITE_BACK
`cpu_req`	CPU → Ctrl	sampled	ignored	ignored	ignored
`cpu_stall`	Ctrl → CPU	0 (→1 on req)	1	1	1
`cpu_dout`	Ctrl → CPU	—	valid on hit	valid when done	—
`hit`	Ctrl (internal)	0	1 if hit	0	0
`mem_read`	Ctrl → Mem	0	0	**1**	0
`mem_write`	Ctrl → Mem	0	0 (→1 miss dirty)	0	**1**

5. Core FSM Tick Logic

5.1. Core FSM Tick Logic

```python
def tick(self, cpu_req):
    self.cycle += 1
    prev = self.state

    # Reset all pulse-mode signals each cycle
    self.hit = False
    self.cpu_dout = None
    self.mem_read = self.mem_write = False

    # Dispatch to handler for current state
    {
        State.IDLE        : self._idle,
        State.COMPARE_TAG : self._compare_tag,
        State.ALLOCATE    : self._allocate,
        State.WRITE_BACK  : self._write_back,
    }[self.state](cpu_req)

    self._record(prev, self._describe(prev))
```

Listing 3: Core `tick()` dispatcher in `CacheController`

Self-Documenting Log

Every call to `tick()` appends a `CycleLog` entry capturing the state transition, all signal values, and a snapshot of the four cache lines. This log is the basis for the detailed per-cycle traces in Section 6.

6. Simulation Scenarios

Six scenarios exercise every state transition of the FSM. Each scenario shows the request sequence, a cycle-by-cycle state trace, the CPU-visible results, and the final cache snapshot.

6.1. Scenario 1: Read Miss – Cold Start

Path: IDLE → COMPARE_TAG → ALLOCATE (×3) → IDLE

With all cache lines invalid (`valid=0`), any request causes a cold miss. This is the simplest path through the FSM: **`IDLE`** receives the request, **`COMPARE_TAG`** detects `valid=0` (immediate miss), **`ALLOCATE`** spends 3 cycles waiting for memory, then fills the line and releases the CPU.

Request: `READ addr=0x05` (tag=0x0, idx=1, off=1)
Memory preloaded: `[0xAA, 0xBB, 0xCC, 0xDD]` at block base `0x04`.

Table 4: Scenario 1 – Cycle Trace

Cycle	State In	→	State Out	HIT	STALL	MRD	MWR	Description
1	IDLE	→	COMPARE_TAG	—	T	F	F	New request latched
2	COMPARE_TAG	→	ALLOCATE	**MISS**	T	T	F	valid=0 → fetch from memory
3	ALLOCATE	→	ALLOCATE	—	T	T	F	Waiting…(countdown=2
4	ALLOCATE	→	ALLOCATE	—	T	T	F	Waiting…(countdown=1
5	ALLOCATE	→	IDLE	—	F	F	F	Block loaded; data = 0xBB served
6	IDLE	→	IDLE	—	F	F	F	Idle

Result: `READ 0x05 → 0xBB` Total cycles: 5 Miss penalty: 4 cycles

Final Cache State:

```
[0]  V=0 D=0  tag=0x0  [0x00, 0x00, 0x00, 0x00]
[1]  V=1 D=0  tag=0x0  [0xAA, 0xBB, 0xCC, 0xDD]  <- loaded; 0xBB
   served
[2]  V=0 D=0  tag=0x0  [0x00, 0x00, 0x00, 0x00]
[3]  V=0 D=0  tag=0x0  [0x00, 0x00, 0x00, 0x00]
```

6.2. Scenario 2: Spatial Locality — Read Miss then Hits

Path: IDLE → CMP_TAG → ALLOCATE → IDLE, then 3× (IDLE → CMP_TAG → IDLE)

After the cold miss fetches the block, three subsequent reads to the *same block* (different offsets) hit immediately in one cycle each, demonstrating spatial locality and block-level fetch amortisation.

Requests: `READ 0x04`, `READ 0x05`, `READ 0x06`, `READ 0x07`
Memory preloaded: `[0x10, 0x20, 0x30, 0x40]` at block base `0x04`.

Table 5: Scenario 2 – Cycle Trace

Cycle	State In	→	State Out	HIT	STALL	MRD	MWR	Description
1–5	IDLE/CT/ALLOC	→	IDLE	**MISS**	T	T	F	Read 0x04: load block
6–7	IDLE	→	IDLE	**HIT**	F	F	F	Read 0x05 → 0x20
8–9	IDLE	→	IDLE	**HIT**	F	F	F	Read 0x06 → 0x30
10–11	IDLE	→	IDLE	**HIT**	F	F	F	Read 0x07 → 0x40

Results: `0x04`→`0x10`, `0x05`→`0x20`, `0x06`→`0x30`, `0x07`→`0x40`
Hit rate after cold miss: 75% (3/4 requests served from cache).

6.3. Scenario 3: Write Hit — Dirty Bit Confirmed by Read-Back

Path: IDLE → CMP_TAG → IDLE (write hit), followed by read-back hit

A write to a cached address updates the word in-place without touching memory, and sets the dirty bit. A subsequent read to the same address returns the newly written value - confirming the in-cache update.

Requests: `READ 0x00` (load block), `WRITE 0x01 ← 0xFF` (write hit), `READ 0x01` (verify)
Memory preloaded: `[0xAA, 0xBB, 0xCC, 0xDD]` at `0x00`.

Table 6: Scenario 3 – Cycle Trace

Cycle	State In	→	State Out	HIT	STALL	MRD	MWR	Description
1–5	IDLE/CT/ALLOC	→	IDLE	**MISS**	T	T	F	Read 0x00: load block
6–7	IDLE	→	IDLE	**HIT**	F	F	F	Write 0x01←0xFF; dirty=1
8–9	IDLE	→	IDLE	**HIT**	F	F	F	Read 0x01 → 0xFF

Results: `WRITE 0x01←0xFF  READ 0x01 → 0xFF`
Memory at `0x01` still holds `0xBB` (write-back policy; not yet evicted).

Final Cache State:

```
[0]  V=1 D=1  tag=0x0  [0xAA, 0xFF, 0xCC, 0xDD]  <- dirty (0xFF
   modified)
```

6.4. Scenario 4: Write Miss — Allocate-on-Write

Path: IDLE → CMP_TAG → ALLOCATE → IDLE (with write applied on fill)

On a write to an address not currently in cache, the controller first fetches the entire block (**`ALLOCATE`**), then immediately applies the write and sets the dirty bit — all before releasing the CPU. This is the "allocate-on-write" policy.

Request: `WRITE 0x09 ← 0x77`
Memory preloaded: `[0x50, 0x60, 0x70, 0x80]` at block base `0x08`.

Table 7: Scenario 4 – Cycle Trace

Cycle	State In	→	State Out	HIT	STALL	MRD	MWR	Description
1	IDLE	→	COMPARE_TAG	—	T	F	F	Write request latched
2	COMPARE_TAG	→	ALLOCATE	**MISS**	T	T	F	valid=0 → fetch block
3–4	ALLOCATE	→	ALLOCATE	—	T	T	F	Waiting...
5	ALLOCATE	→	IDLE	—	F	F	F	Fill; apply write; dirty=1

Result: Block fetched; `0x09` updated to `0x77`; `dirty=1`
Final Cache: `[2] V=1 D=1 tag=0x0 [0x50, 0x60, 0x77, 0x80]`

6.5. Scenario 5: Dirty Eviction — WRITE_BACK Path

Path: ... → CMP_TAG → WRITE_BACK (×3) → ALLOCATE (×3) → IDLE

This scenario demonstrates the worst-case eviction path. A dirty block in index 0 must be written back to memory before the new conflicting block can be fetched. The total miss penalty spans 7 cycles.

Requests:

1. `READ 0x00` — cold miss; loads `[0x11, 0x22, 0x33, 0x44]` into index 0
2. `WRITE 0x01 ← 0x99` — write hit; index 0 becomes dirty
3. `READ 0x40` — tag=0x4, index 0 — **conflict miss on dirty line**; old block must be evicted

Table 8: Scenario 5 – Cycle Trace

Cyc	State In	→	State Out	HIT	ST	MR	MW	Description
1	IDLE	→	COMPARE_TAG	—	T	F	F	Latch READ 0x00
2	COMPARE_TAG	→	ALLOCATE	**MISS**	T	T	F	Cold miss
3–4	ALLOCATE	→	ALLOCATE	—	T	T	F	Waiting…
5	ALLOCATE	→	IDLE	—	F	F	F	Block loaded
6	IDLE	→	COMPARE_TAG	—	T	F	F	Latch WRITE 0x01
7	COMPARE_TAG	→	IDLE	**HIT**	F	F	F	Write hit; dirty=1
8	IDLE	→	COMPARE_TAG	—	T	F	F	Latch READ 0x40
9	COMPARE_TAG	→	WRITE_BACK	**MISS**	T	F	T	tag=0x4≠0x0 + dirty
10–11	WRITE_BACK	→	WRITE_BACK	—	T	F	T	Writing dirty block…
12	WRITE_BACK	→	ALLOCATE	—	T	T	F	WB done; fetch 0x40
13–14	ALLOCATE	→	ALLOCATE	—	T	T	F	Waiting…
15	ALLOCATE	→	IDLE	—	F	F	F	Block loaded; dout=0xAA

Dirty Eviction Penalty

The total miss penalty for `READ 0x40` spans 7 cycles (cycles 9–15): 3 for writing back the old block + 3 for reading the new block + 1 for the **`COMPARE_TAG`** cycle. This is the worst case for our FSM.

Results: `READ 0x00` → `0x11` (miss), `WRITE 0x01`←`0x99` (hit), `READ 0x40` → `0xAA` (miss+WB)

Final Cache:

```
[0]  V=1 D=0  tag=0x4  [0xAA, 0xBB, 0xCC, 0xDD]  <- tag changed 0x0
   -> 0x4
     (dirty block [0x11,0x99,0x33,0x44] flushed to mem[0x00..0x03])
```

6.6. Scenario 6: Mixed Read/Write Stress Test

7 requests · 30 cycles · all state transitions exercised

This scenario chains all previously demonstrated behaviours into a realistic mixed workload. Four different cache lines are populated, writes dirty two of them, and a final aliased read forces one dirty eviction through the **`WRITE_BACK`** path.

Request Sequence:

1. `READ 0x04` — (idx=1, tag=0) — **MISS**; loads block

2. `WRITE 0x05 ← 0xBE` — **HIT** on same block; dirty
3. `READ 0x08` — (idx=2, tag=0) — **MISS**; loads block
4. `READ 0x04` — (idx=1) — **HIT**; returns 0x10
5. `WRITE 0x08 ← 0xEF` — **HIT** on idx=2; dirty
6. `READ 0x0C` — (idx=3, tag=0) — **MISS**; loads block
7. `READ 0x44` — (idx=1, tag=4) — **alias: DIRTY eviction of 0x04's block!**

Table 9: Scenario 6 – Cycle Trace

Cyc	State In	→	State Out	HIT	ST	MR	MW	Description
1–5	CT/ALLOC	→	IDLE	**MISS**	T	T	F	Read 0x04: load into [1]
6–7	IDLE	→	IDLE	**HIT**	F	F	F	Write 0x05←0xBE; [1] dirty
8–12	CT/ALLOC	→	IDLE	**MISS**	T	T	F	Read 0x08: load into [2]
13–14	IDLE	→	IDLE	**HIT**	F	F	F	Read 0x04: HIT [1]→0x10
15–16	IDLE	→	IDLE	**HIT**	F	F	F	Write 0x08←0xEF; [2] dirty
17–21	CT/ALLOC	→	IDLE	**MISS**	T	T	F	Read 0x0C: load into [3]
22	IDLE	→	COMPARE_TAG	—	T	F	F	Latch READ 0x44
23	CMP_TAG	→	WRITE_BACK	**MISS**	T	F	T	tag=4≠0 + dirty; evict [1]
24–25	WRITE_BACK	→	WRITE_BACK	—	T	F	T	Writing dirty block...
26	WRITE_BACK	→	ALLOCATE	—	T	T	F	WB done; fetch 0x44 block
27–28	ALLOCATE	→	ALLOCATE	—	T	T	F	Waiting...
29	ALLOCATE	→	IDLE	—	F	F	F	Block loaded; 0xDE served

Results:

- `READ 0x04 → 0x10`
- `WRITE 0x05 ← 0xBE`
- `READ 0x08 → 0x50`
- `READ 0x04 → 0x10 (HIT)`
- `WRITE 0x08 ← 0xEF`
- `READ 0x0C → 0x90`
- `READ 0x44 → 0xDE (WB eviction)`

Final Cache State:

```
[0]  V=0 D=0  tag=0x0  [0x00, 0x00, 0x00, 0x00]
[1]  V=1 D=0  tag=0x4  [0xDE, 0xAD, 0xBE, 0xEF]  <- new block (dirty
    [1] evicted)
[2]  V=1 D=1  tag=0x0  [0xEF, 0x60, 0x70, 0x80]  DIRTY
[3]  V=1 D=0  tag=0x0  [0x90, 0xA0, 0xB0, 0xC0]
```

7. Conclusion

The simulation faithfully reproduces the four-state cache controller FSM described in Patterson & Hennessy. All six scenarios produce cycle-accurate, verified results:

- **Cold read miss** - clean **`ALLOCATE`** path, 5 cycles total, correct data returned.
- **Spatial locality hits** - block fetch amortised over 4 reads; 75% hit rate demonstrated.
- **Write hit / dirty bit** - in-cache update confirmed by subsequent read returning modified value; memory unchanged until eviction.
- **Write miss / allocate-on-write** - block fetched transparently before write; dirty bit set immediately.
- **Dirty eviction** - **`WRITE_BACK`** state correctly reconstructs old block address, flushes to memory, then fetches new block; total penalty 7 cycles.
- **Mixed stress test** - all state transitions combined in a realistic sequence; 30 cycles, all results correct.

The simulation architecture cleanly separates the three components (CPU, FSM controller, memory), enabling independent testing and easy extension. Possible extensions include: set-associative mappings with LRU/FIFO replacement, a write-through no-allocate policy, multi-level caches, or integration with a simple pipeline model.